# Module 5: Regression and Classification Trees

Xiao Wang

Department of Statistics
Purdue University

# Table of contents

1. Introduction

2. Regression Tree

3. Classification Tree

4. Bagging and Random Forest

# Introduction

# Tree-based methods

- Here we describe tree-based methods for regression and classification.
- These involve stratifying or segmenting the predictor space into a number of simple regions.
- Since the set of splitting rules used to segment the predictor space can be summarized in a tree, these types of approaches are known as decision-tree methods.

# Pros and Cons

- Tree-based methods are simple and useful for interpretation.
- However they typically are not competitive with the best supervised learning approaches in terms of prediction accuracy.
- Hence we also discuss bagging, random forests, and boosting. These methods grow multiple trees which are then combined to yield a single consensus prediction.
- Combining a large number of trees can often result in dramatic improvements in prediction accuracy, at the expense of some loss interpretation.

# The basics of decision trees

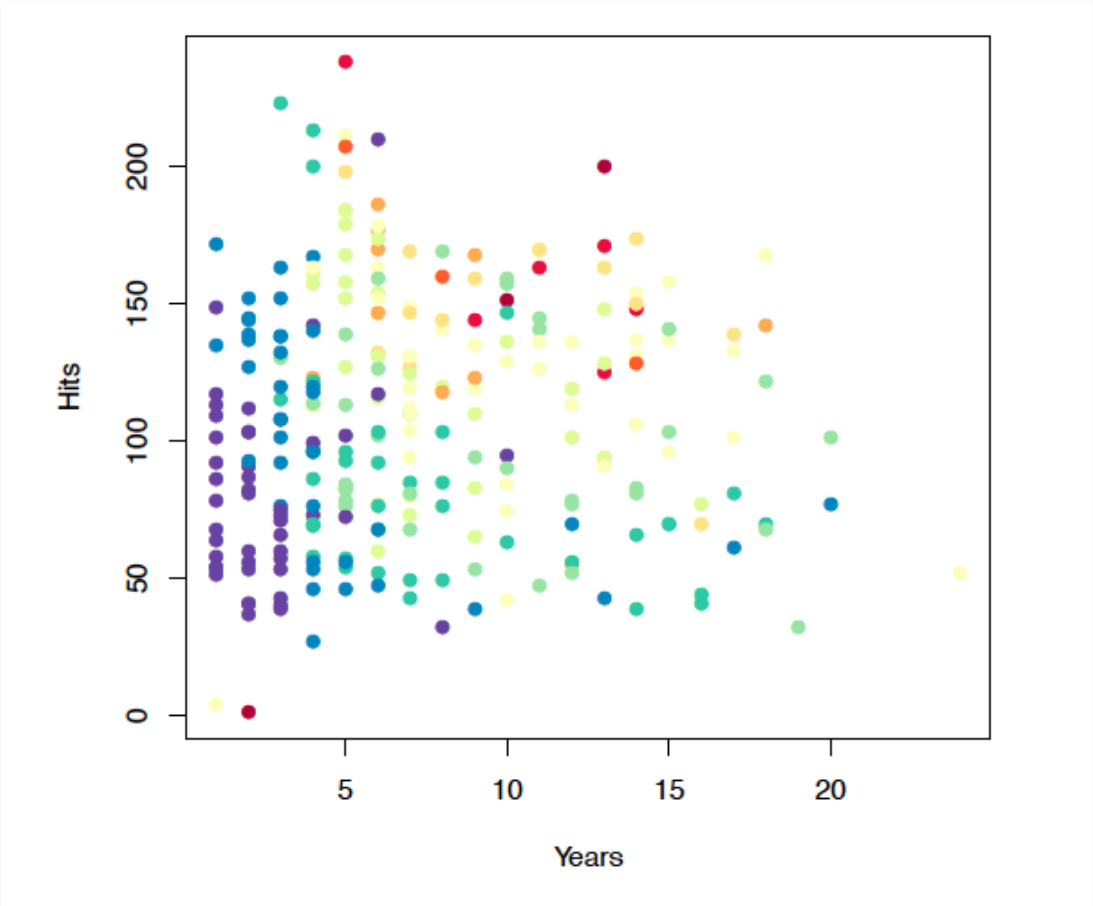
- Decision trees can be applied to both regression and classification problems.
- We first consider regression problems, and then move on to classification.

# Regression Tree

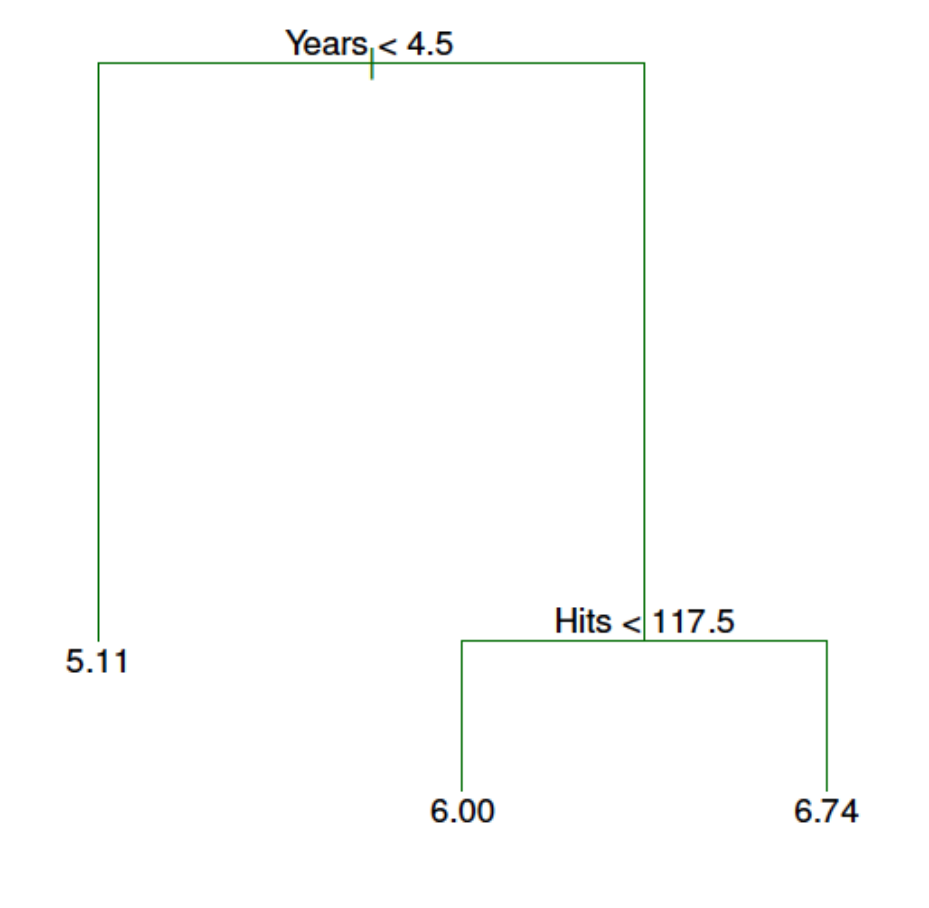
# Baseball salary data

Salary is color-coded from low (blue, green) to high (yellow,red)

How would you stratify it?



# Decision tree for these data

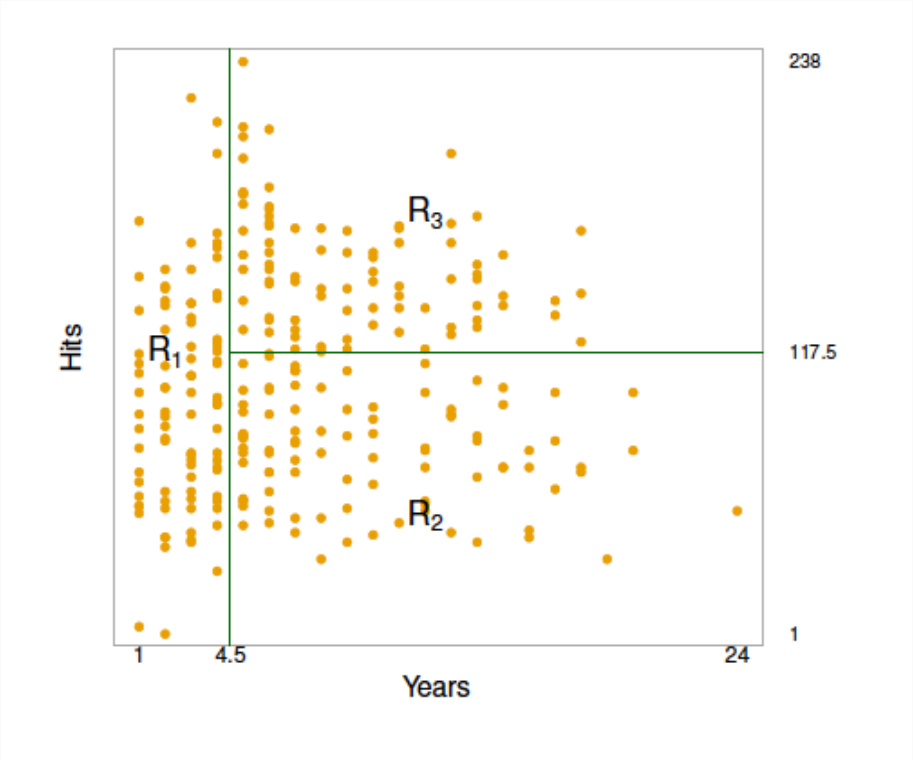


# Details of previous figure

- For the Hitters data, a regression tree for predicting the log salary of a baseball player, based on the number of years that he has played in the major leagues and the number of hits that he made in the previous year.
- At a given internal node, the label (of the form $X_j < t_k$) indicates the left-hand branch emanating from that split, and the right-hand branch corresponds to $X_j \geq t_k$. For instance, the split at the top of the tree results in two large branches. The left-hand branch corresponds to $\mathit{Years} < 4.5$, and the right-hand branch corresponds to $\mathit{Years} >= 4.5$.
- The tree has two internal nodes and three terminal nodes, or leaves. The number in each leaf is the mean of the response for the observations that fall there.

# Results

Overall, the tree stratifies or segments the players into three regions of predictor space: $R_1 = \{X | Years < 4.5\}$, $R_2 = \{X | Years \geq 4.5, Hits < 117.5\}$, an $R_3 = \{X | Years \geq 4.5, Hits \geq 117.5\}$.



# Terminology for trees

- In keeping with the tree analogy, the regions $R_1$, $R_2$, and $R_3$ are known as terminal nodes
- Decision trees are typically drawn upside down, in the sense that the leaves are at the bottom of the tree.
- The points along the tree where the predictor space is split are referred to as internal nodes
- In the hitters tree, the two internal nodes are indicated by the text $\mathit{Years} < 4.5$ and $\mathit{Hits} < 117.5$.

# Interpretation of results

- Years is the most important factor in determining Salary, and players with less experience earn lower salaries than more experienced players.
- Given that a player is less experienced, the number of Hits that he made in the previous year seems to play little role in his Salary.
- But among players who have been in the major leagues for five or more years, the number of Hits made in the previous year does affect Salary, and players who made more Hits last year tend to have higher salaries.
- Surely an over-simplification, but compared to a regression model, it is easy to display, interpret and explain

# Details of the tree-building process

- We divide the predictor space — that is, the set of possible values for $X_1, X_2, \ldots, X_p$ — into $J$ distinct and non-overlapping regions, $R_1, R_2, \ldots, R_J$.
- For every observation that falls into the region $R_j$, we make the same prediction, which is simply the mean of the response values for the training observations in $R_j$.

# More details of the tree-building process

- In theory, the regions could have any shape. However, we choose to divide the predictor space into high-dimensional rectangles, or boxes, for simplicity and for ease of interpretation of the resulting predictive model.
- The goal is to find boxes $R_1, \ldots, R_J$ that minimize the RSS, given by

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2,$$

where $\hat{y}_{R_j}$ is the mean response for the training observations within the $j$th box.

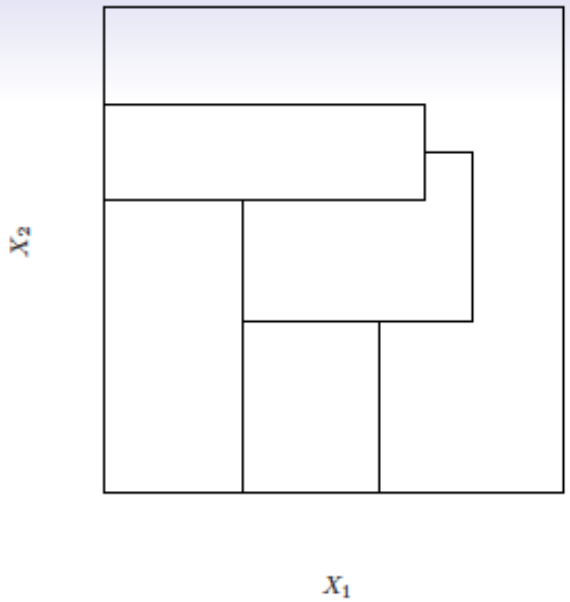
- Unfortunately, it is computationally infeasible to consider every possible partition of the feature space into J boxes.
- For this reason, we take a top-down, greedy approach that is known as recursive binary splitting.
- The approach is top-down because it begins at the top of the tree and then successively splits the predictor space; each split is indicated via two new branches further down on the tree.
- It is greedy because at each step of the tree-building process, the best split is made at that particular step, rather than looking ahead and picking a split that will lead to a better tree in some future step.

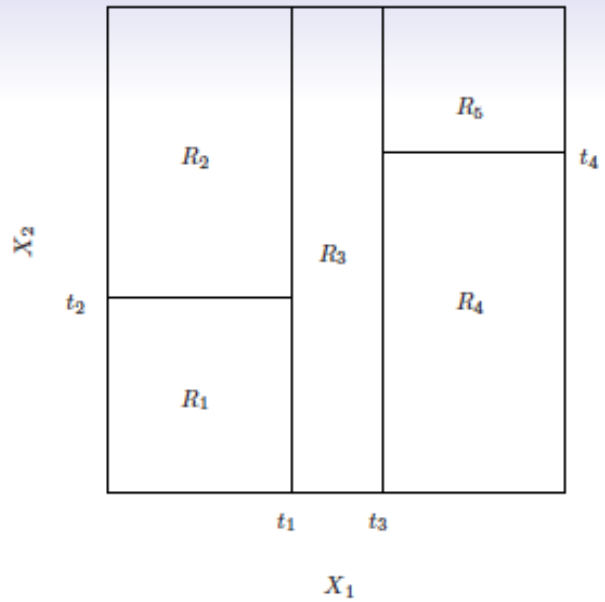
# Details continued

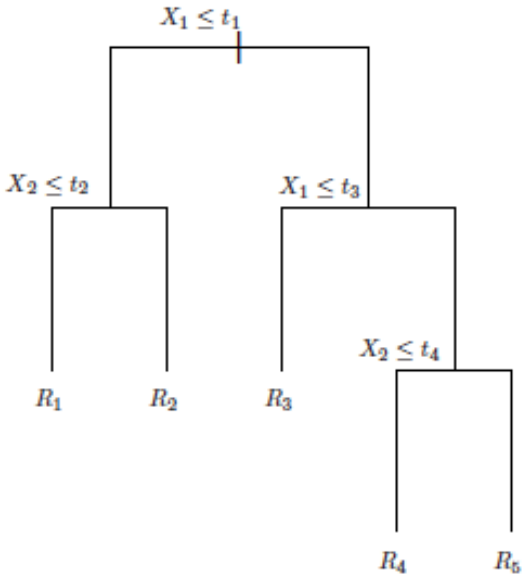
- We first select the predictor $X_j$ and the cutpoint $s$ such that spliting the predictor space into the regions $\{X|X_j < s\}$ and $\{X|X_j \geq s\}$ leads to the greatest possible reduction in RSS.
- Next, we repeat the process, looking for the best predictor and best cutpoint in order to split the data further so as to minimize the RSS within each of the resulting regions.
- However, this time, instead of splitting the entire predictor space, we split one of the two previously identified regions. We now have three regions.
- Again, we look to split one of these three regions further, so as to minimize the RSS. The process continues until a stopping criterion is reached; for instance, we may continue until no region contains more than five observations.

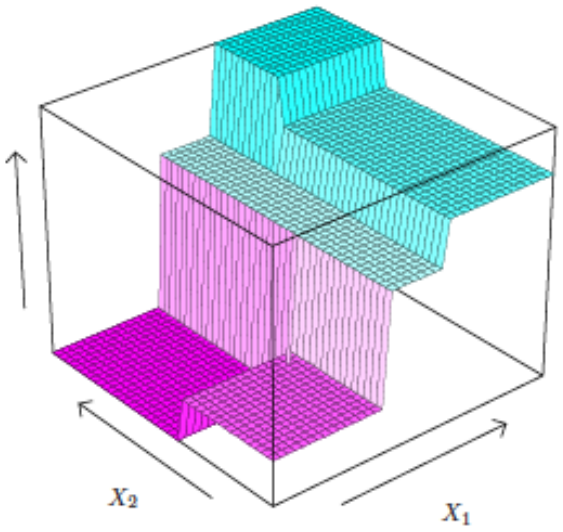
# Predictions

- We predict the response for a given test observation using the mean of the training observations in the region to which that test observation belongs.
- A five-region example of this approach is shown in the next slide.


$X_2$
$X_1$


$R_5$
$R_2$
$t_4$
$X_2$
$R_3$
$R_4$
$t_2$
$R_1$
$t_1$
$t_3$
$X_1$


$X_1 \leq t_1$
$X_2 \leq t_2$
$X_1 \leq t_3$
$X_2 \leq t_4$
$R_1$
$R_2$
$R_3$
$R_4$
$R_5$


$X_2$
$X_1$

# Details of previous figure

Top Left: A partition of two-dimensional feature space that could not result from recursive binary splitting.

Top Right: The output of recursive binary splitting on a two-dimensional example.

Bottom Left: A tree corresponding to the partition in the top right panel.

Bottom Right: A perspective plot of the prediction surface corresponding to that tree.

# Pruning a tree

- The process described above may produce good predictions on the training set, but is likely to overfit the data, leading to poor test set performance.Why?
- A smaller tree with fewer splits (that is, fewer regions $R_1, \ldots, R_J$) might lead to lower variance and better interpretation at the cost of large bias.
- One possible alternative to the process described above is to grow the tree only so long as the decrease in the RSS due to each split exceeds some (high) threshold.
- This strategy will result in smaller trees, but is too short-sighted: a seemingly worthless split early on in the tree might be followed by a very good split — that is, a split that leads to a large reduction in RSS later on.

# Pruning a tree — continued

- A better strategy is to grow a very large tree $T_0$, and then prune it back in order to obtain a subtree
- Cost complexity pruning — also known as weakest link pruning — is used to do this
- We consider a sequence of trees indexed by a nonnegative tuning parameter $\alpha$. For each value of $\alpha$ there corresponds a subtree $T \subset T_0$ such that

$$\sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

  is as small as possible. Here $|T|$ indicates the number of terminal nodes of the tree $T$, $R_m$ is the rectangle (i.e. the subset of predictor space) corresponding to the $m$th terminal node, and $\hat{y}_{R_m}$ is the mean of the training observations in $R_m$.

# Choosing the best subtree

- The tuning parameter $\alpha$ controls a trade-off between the subtree's complexity and its fit to the training data.
- We select an optimal value $\hat{\alpha}$ using cross-validation.
- We then return to the full data set and obtain the subtree corresponding to $\hat{\alpha}$.

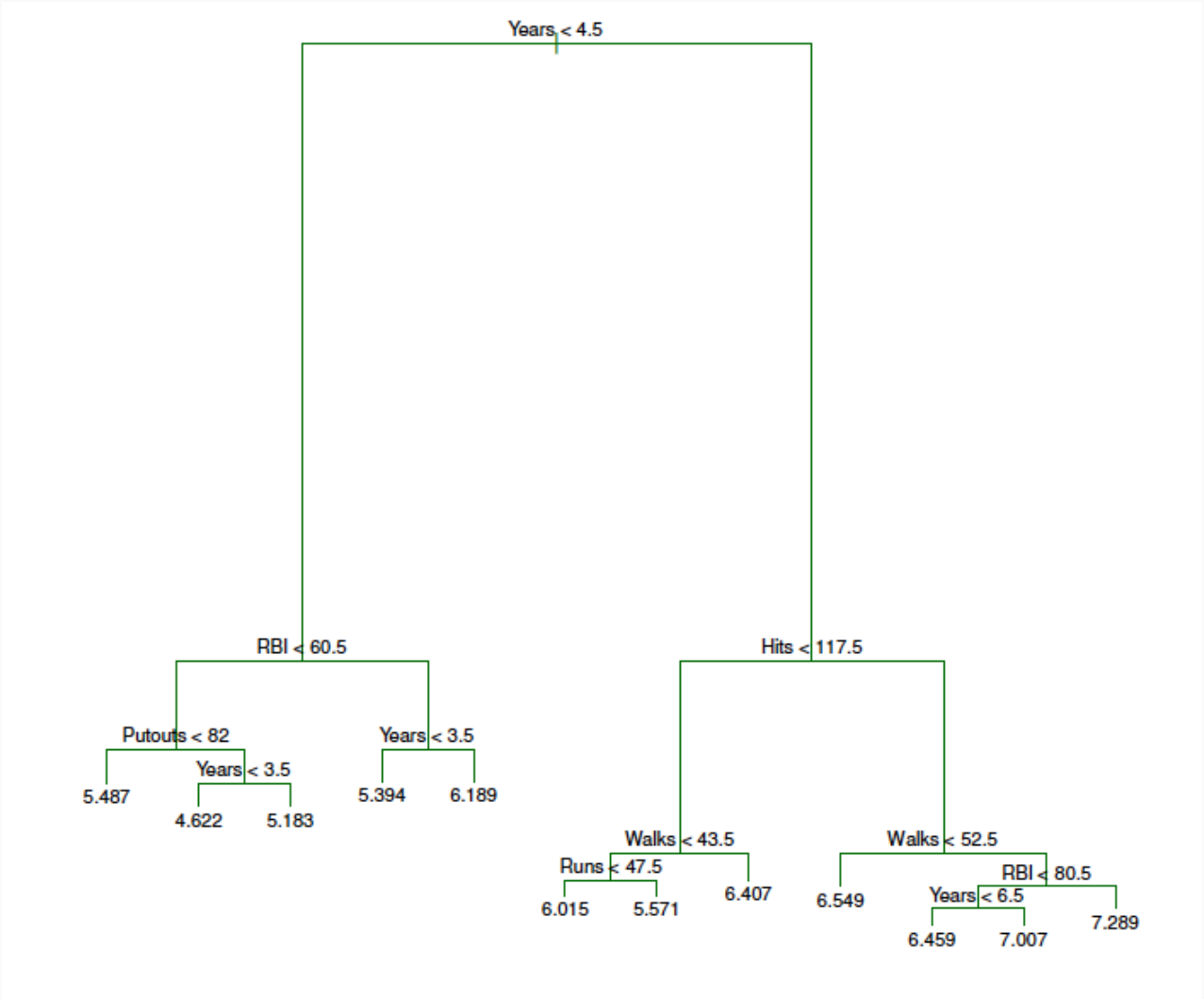
# Summary: tree algorithm

1. Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum number of observations.
2. Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of $\alpha$.
3. Use K-fold cross-validation to choose $\alpha$. For each $k = 1, \ldots, K$:
   — Repeat Steps 1 and 2 on the $(K-1)/K$th fraction of the training data, excluding the $k$th fold.
   — Evaluate the mean squared prediction error on the data in the left-out $k$th fold, as a function of $\alpha$.
   Average the results, and pick $\alpha$ to minimize the average error.
4. Return the subtree from Step 2 that corresponds to the chosen value of $\alpha$.

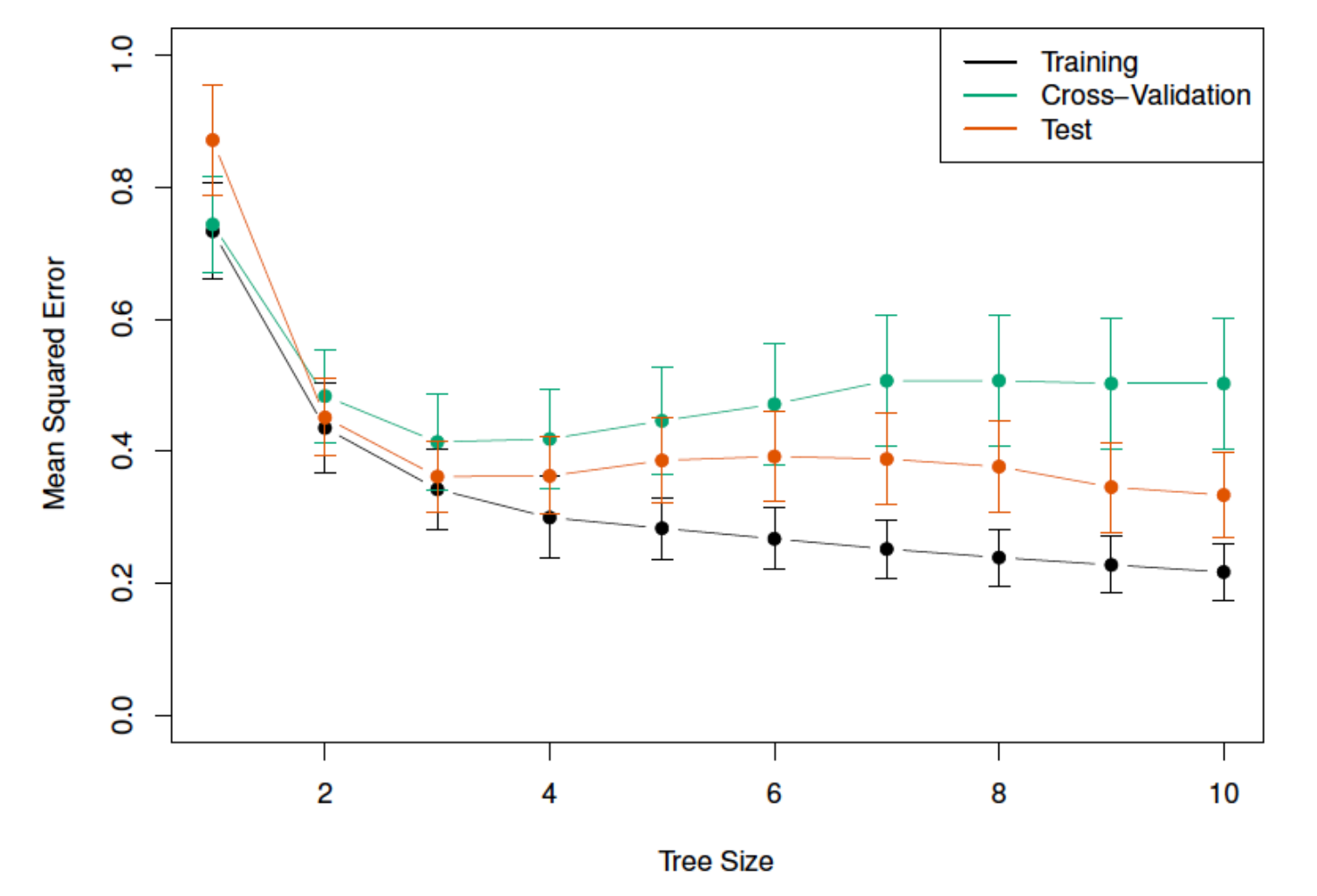
# Baseball example continued

- First, we randomly divided the data set in half, yielding 132 observations in the training set and 131 observations in the test set.
- We then built a large regression tree on the training data and varied $\alpha$ in in order to create subtrees with different numbers of terminal nodes.
- Finally, we performed six-fold cross-validation in order to estimate the cross-validated MSE of the trees as a function of $\alpha$.

# Baseball example continued



# Baseball example continued

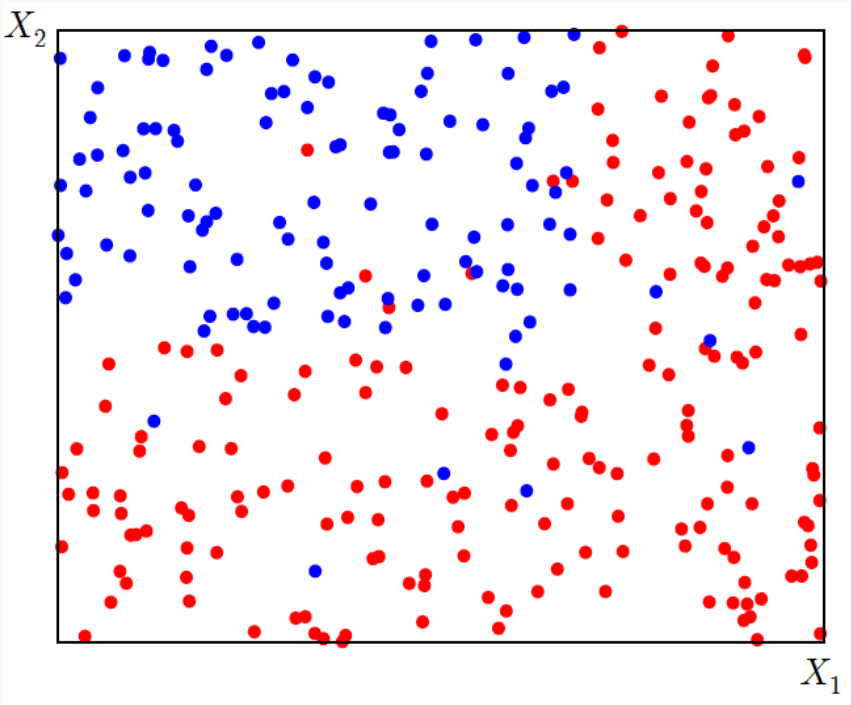


# Classification Tree

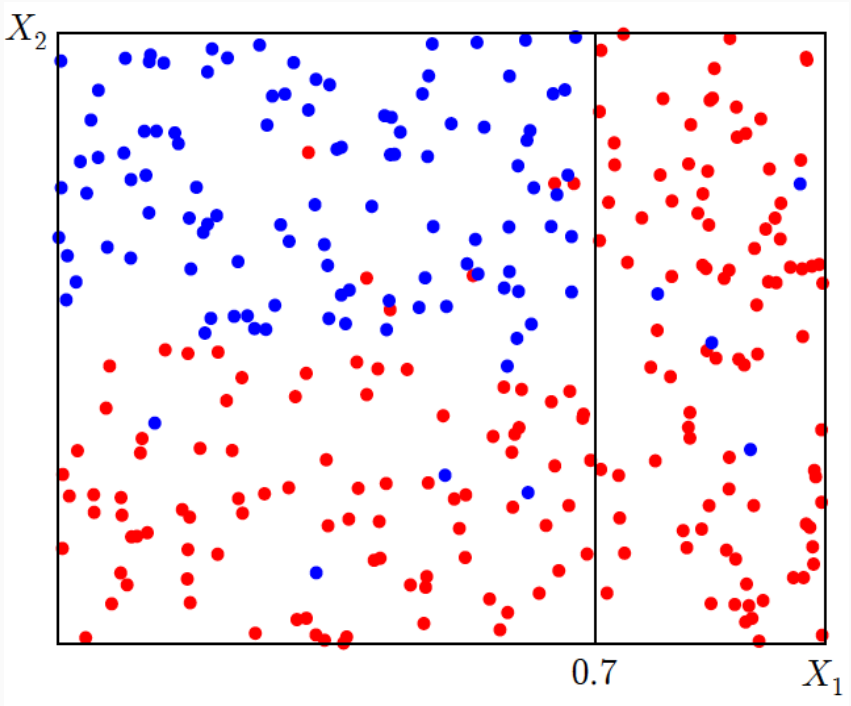
# Classification tree

- Very similar to a regression tree, except that it is used to predict a qualitative response rather than a quantitative one.
- For a classification tree, we predict that each observation belongs to the most commonly occurring class of training observations in the region to which it belongs.

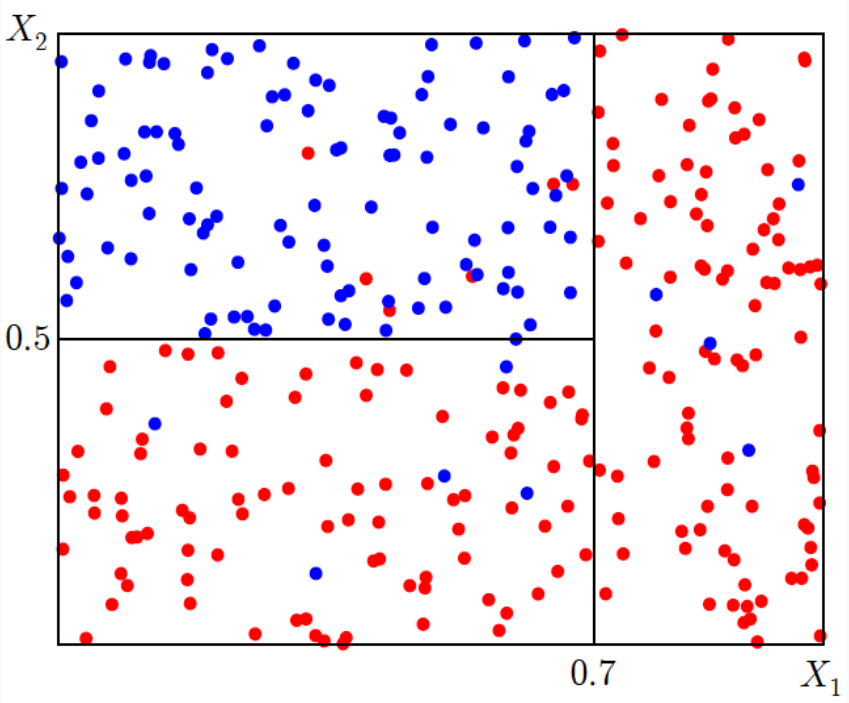
# An Example



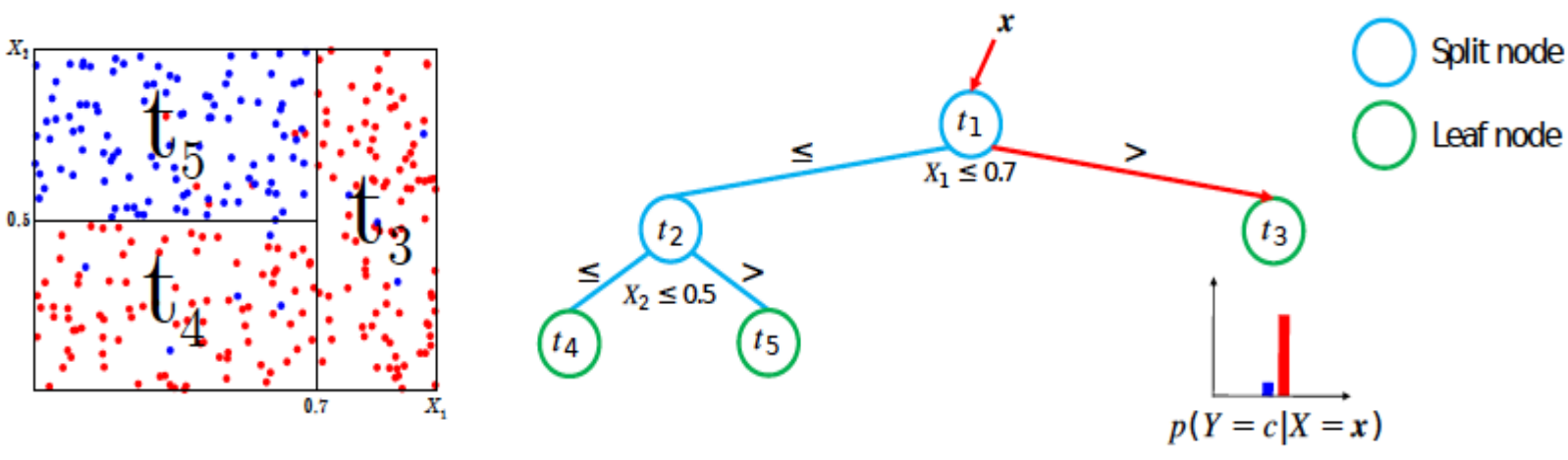
# An Example



# An Example



# An Example




$t \in \varphi$ : nodes of the tree $\varphi$

$X_t$ : split variable at $t$

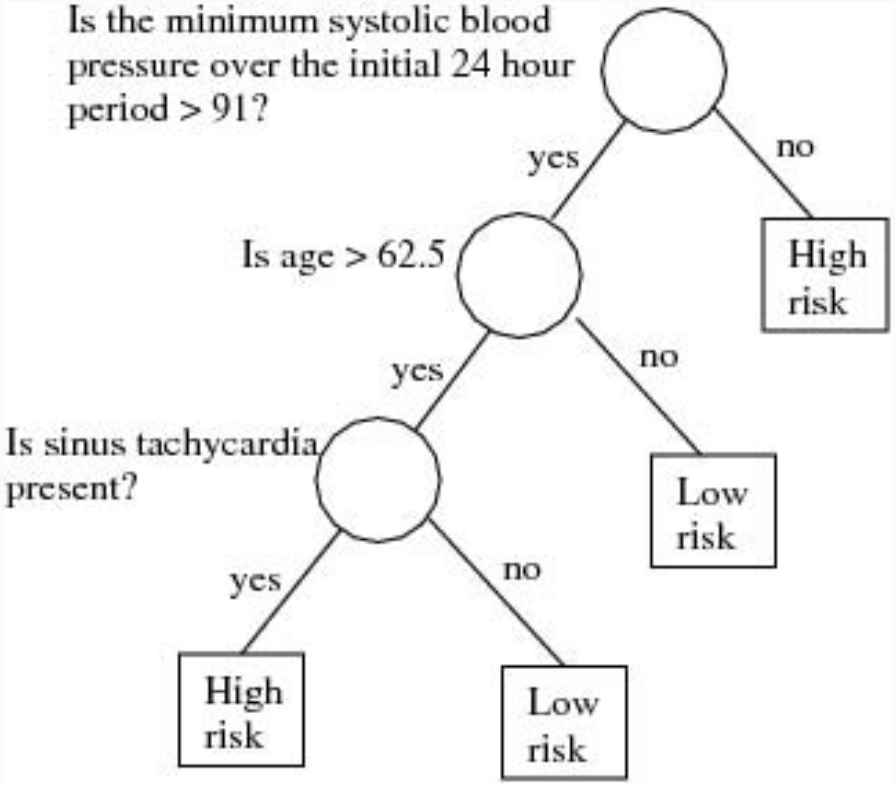
$v_t \in \mathbb{R}$ : split threshold at $t$

$\varphi(\mathbf{x}) = \arg\max_{c \in y} p(Y = c|X = \mathbf{x})$

# Tree structure

- Reference: *Classification and Regression Trees* by L. Breiman, J. Friedman, R. Olshen, and C. Stone, Chapman & Hall, 1984
- A medical example:
  - Predict high risk patients who will not survive at least 30 days on the basis of the initial 24-hour data
  - 19 variables are measured during the first 24 hours. These include blood pressure, age, etc

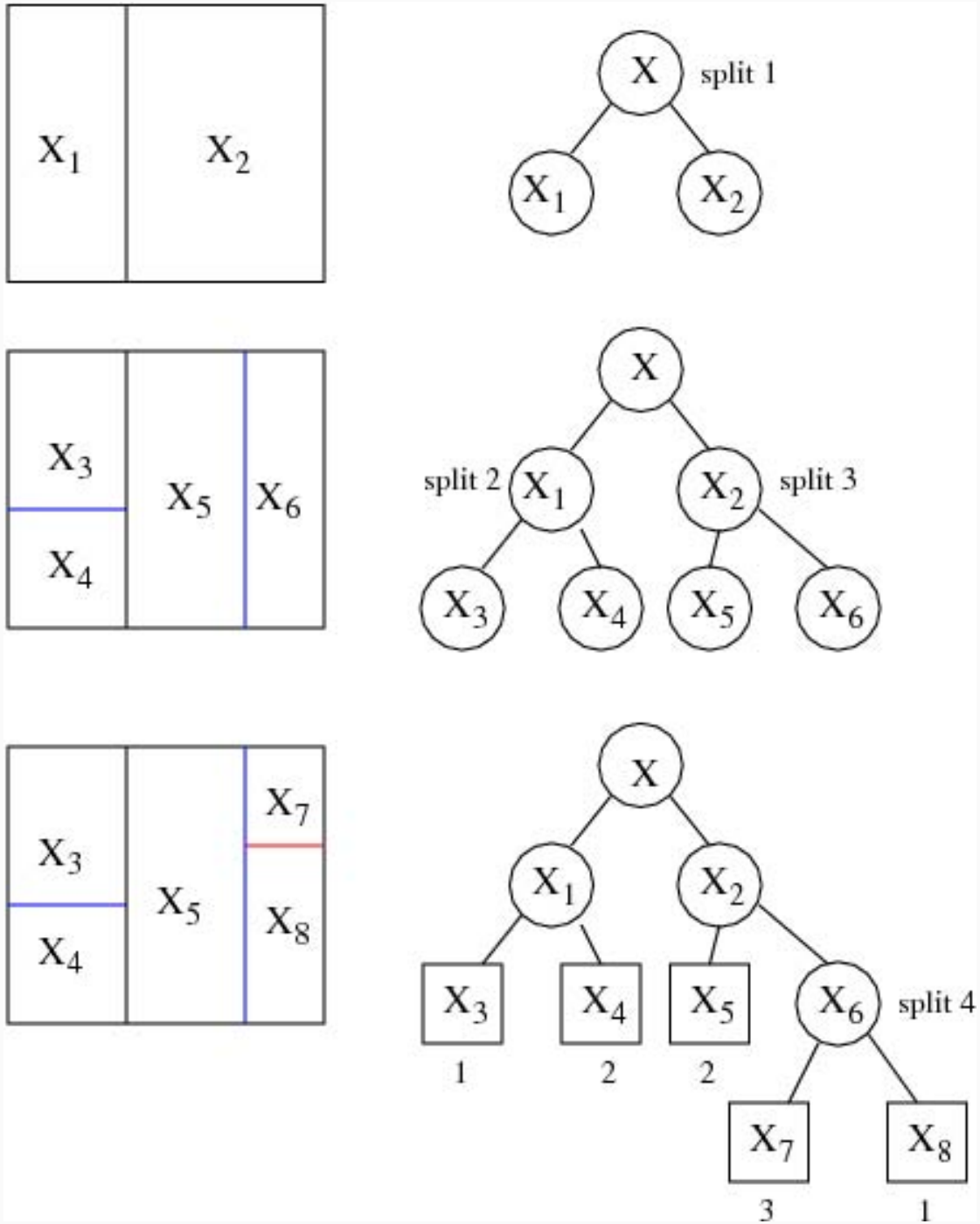
# An example of tree structure



Some notations:

- Denote the feature space by $\mathcal{X}$. The covariate $X = (X_1, X_2, \ldots, X_p)^T \in \mathcal{X} \subset \mathbb{R}^p$, some of which may be categorical
- Classification trees are constructed by repeated splits of subsets of $\mathcal{X}$ into two descendant subsets, beginning with $\mathcal{X}$ itself
- Definitions: node, terminal node (leaf node), parent node, child node, sibling node
- Two child nodes forms a partition of the region occupied by their parent node
- Every leaf node is assigned with a class

- A node is denoted by $t$
- Its left child node is denoted by $t_L$ and right by $t_R$
- The collection of all the nodes is denoted by $T$
- The collection of all the leaf nodes by $\widetilde{T}$
- A split is denoted by $s$, and the set of splits is denoted by $\mathcal{S}$


X1
X2
X
split 1
X1
X2
X3
X4
X5
X6
X
split 2
X1
X2
split 3
X3
X4
X5
X6
X7
X3
X5
X8
X4
X
X1
X2
X3
X4
X5
X6
split 4
1
2
2
X7
X8
3
1

# Key elements of a tree

- The construction of a tree involves the following three elements:
  1. The decisions when to declare a node terminal or to continue splitting it
  2. The selection of the splits
  3. The assignment of each terminal node to a class
- In particular,
  1. A set of binary questions used as splits
  2. A goodness of split criterion $\phi(s, t)$ that can be evaluated for any split $s$ of any node $t$
  3. A stop-splitting rule
  4. A rule for assigning every terminal node to a class

# Standard set of questions

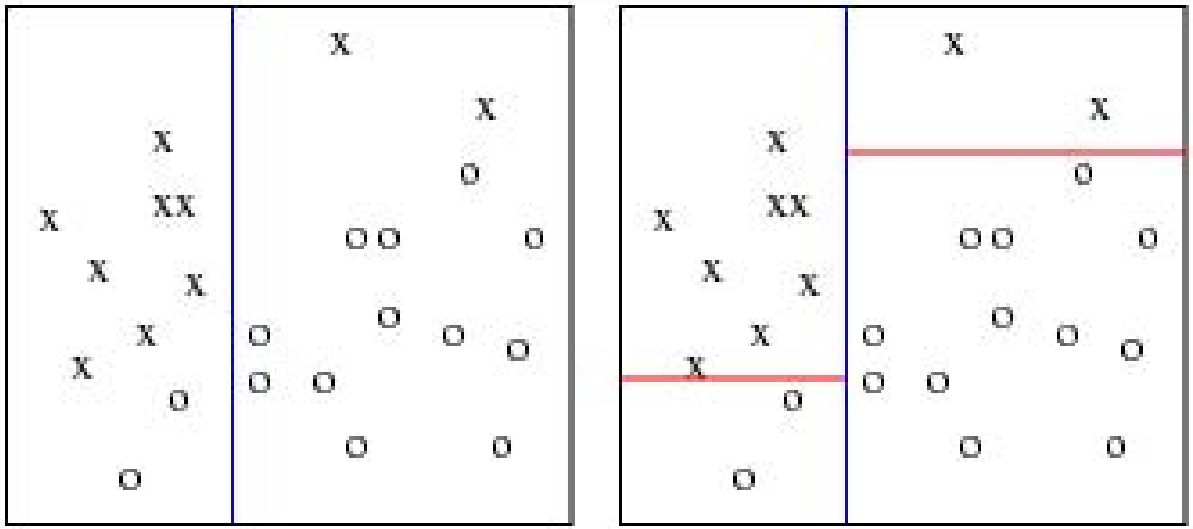
- Each split depends on the value of only a unique variable
- For each variable $X_j$, $\mathcal{S}$ includes all questions of the form

$$\{\text{Is } X_j \in A?\}$$

  - If $X_j$ is continuous or ordinal, $A = (-\infty, c]$
  - If $X_j$ is categorical, $A$ is a subset of categories
- The splits for all $p$ variables constitute the standard set of questions

# Goodness of splits

- The goodness of split is measured by an impurity function defined for each node
- Intuitively, we want each leaf node to be "pure", that is, one class dominates



# The impurity function

Define $p_{tk} = P(Y = k|t) = |t|^{-1} \sum_{i \in t} I(y_i = k)$, possible impurity functions $i(t)$ are

- Misclassification error: $1 - \max_k p_{tk}$
- Gini index: $\sum p_{tk}(1 - p_{tk}) = 1 - \sum p_{tk}^2$
- Cross entropy: $-\sum p_{tk} \log p_{tk}$

2 red & 2 blue balls:

$$gini = 1 - 0.5^2 - 0.5^2 = 0.5$$

3 red & 1 blue balls:

$$gini = 1 - 0.75^2 - 0.25^2 = 0.375$$

4 red & 0 blue balls:

$$gini = 1 - 1^2 - 0^2 = 0$$

# Splitting the data

The quality of splitting data is very important during the training phase. When splitting, we choose to partition the data by the attribute that results in the smallest impurity of the new nodes.

The information gain is the difference between a parent node's entropy and the weighted sum of its child node entropies.

Goodness of split: $\phi(s, t) = i(t) - p_L i(t_L) - p_R i(t_R)$, where $p_L$ and $p_R$ are proportions of the samples in node $t$ that go to the left child $t_L$ and the right child of $t_R$, respectively

We want to determine the attribute that offers the highest Information Gain.

By splitting the data to minimize the impurity scores of the resulting nodes, we get a precise tree.

# Stopping criteria

- A simple criteria: stop splitting a node $t$ when

$$\max_{s \in \mathcal{S}} \frac{|t|}{n} \phi(s, t) < \beta,$$

  where $\beta$ is a pre-specified threshold
- The idea is to stop splitting when the node is sufficiently pure or sufficiently small
- The above stopping criteria may not work well; other more sophisticated stopping rules have been proposed

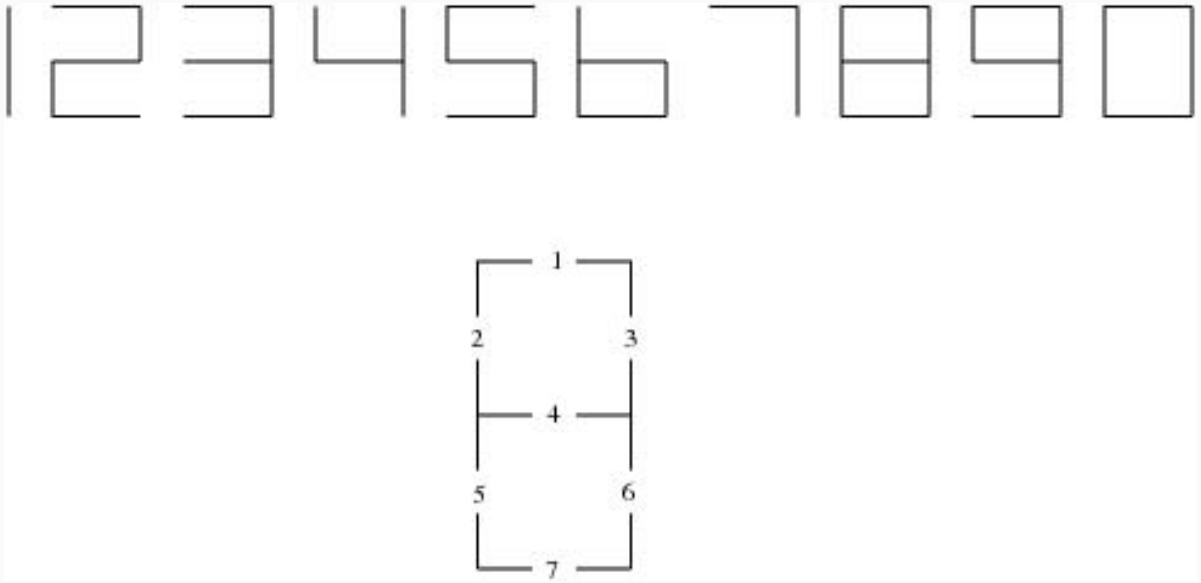
# Class assignment rule

- A class assignment rule assigns a class $k = \{1, \ldots, K\}$ to every terminal node $t \in \widetilde{T}$
- The class assigned to node $t \in \widetilde{T}$ is denoted by $\kappa(t)$
- For 0-1 loss, the class assignment rule is

$$\kappa(t) = \underset{k}{\operatorname{argmax}} \ \hat{p}_{tk}$$

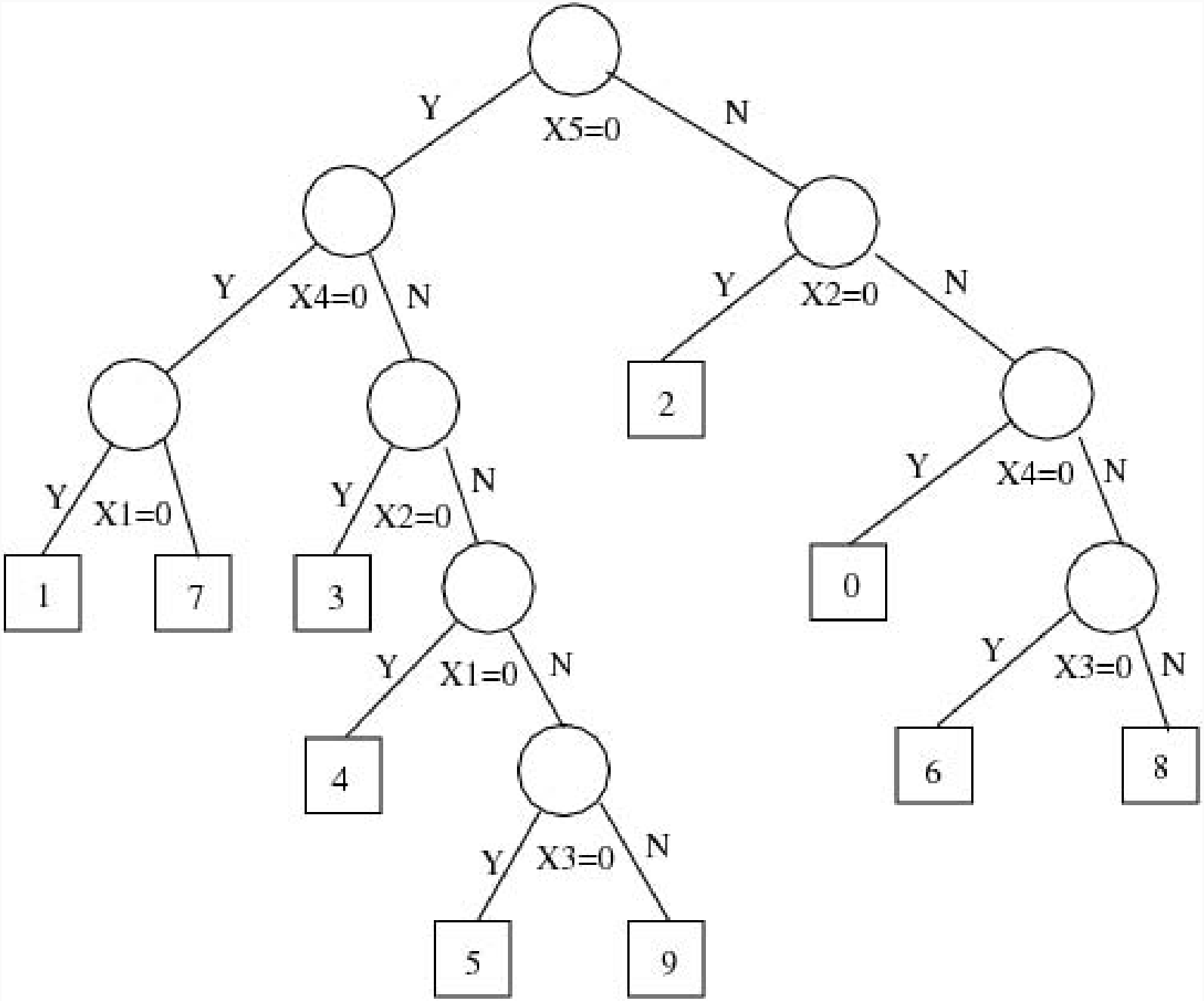
# Example: Digit recognition example

- The 10 digits are shown by different on-off combinations of seven horizontal and vertical lights
- Each digit can be represented by a 7-dimensional vector of zeros and ones. The $i$-th sample is $x_i = (x_{i1}, \ldots, x_{i7})$. If $x_{ij} = 1$, the $j$-th light is on, and vice versa



| Digit | $x_{\cdot 1}$ | $x_{\cdot 2}$ | $x_{\cdot 3}$ | $x_{\cdot 4}$ | $x_{\cdot 5}$ | $x_{\cdot 6}$ | $x_{\cdot 7}$ |
|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
| 2 | 1 | 0 | 1 | 1 | 1 | 0 | 1 |
| 3 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 4 | 0 | 1 | 1 | 1 | 0 | 1 | 0 |
| 5 | 1 | 1 | 0 | 1 | 0 | 1 | 1 |
| 6 | 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| 7 | 1 | 0 | 1 | 0 | 0 | 1 | 0 |
| 8 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 9 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 |

- The data for the example are generated by a malfunctioning calculator
- Each of the seven lights has probability 0.1 of being in the wrong state independently
- The training set contains 200 samples generated according to the specified distribution


X5=0
Y
N
X4=0
Y
N
X2=0
Y
N
X1=0
Y
X2=0
Y
N
X4=0
Y
N
X1=0
Y
N
X3=0
Y
N
X3=0
Y
N
1
7
3
4
5
9
2
0
6
8

# Right-sized trees

- Denote the misclassification error of a tree $T$ by $R(T)$, then

$$R(T) = \sum_{t \in \tilde{T}} p(t) i(t),$$

  where $p(t)$ is the proportion of observations in node $t$, and $i(t)$ is defined by the misclassification error

- $R(T)$ is biased downward, as

$$p(t) i(t) \geq p(t_L) i(t_L) + p(t_R) i(t_R),$$

  thus the larger the tree is, the smaller the misclassification error (overfitting!)

# Digit recognition example: cont'd

| No. Terminal Nodes | $R(T)$ | $R^{ts}(T)$ |
|---|---|---|
| 71 | .00 | .42 |
| 63 | .00 | .40 |
| 58 | .03 | .39 |
| 40 | .10 | .32 |
| 34 | .12 | .32 |
| 19 | .29 | .31 |
| 10 | .29 | .30 |
| 9 | .32 | .34 |
| 7 | .41 | .47 |
| 6 | .46 | .54 |
| 5 | .53 | .61 |
| 2 | .75 | .82 |
| 1 | .86 | .91 |

# Pruning

- Grow a very large tree $T_{max}$
  1. Until all terminal nodes are pure (contain only one class) or contain only identical measurement vectors
  2. When the number of data in each terminal node is no greater than a certain threshold
  3. As long as the tree is sufficiently large

- A "selective" pruning procedure is needed
  1. The pruning is optimal in a certain sense
  2. The search for different ways of pruning should be of manageable computational load

# Minimal cost pruning

For any subtree $T$ of $T_{max}$, define its cost function as

$$R_\lambda(T) = R(T) + \lambda|\widetilde{T}|,$$

where $\lambda$ is a tuning parameter. For each $\lambda$, solve

$$T(\lambda) = \underset{T}{\operatorname{argmin}}\, R_\lambda(T)$$

Remarks:

- To optimally determine $\lambda$, a model selection criterion such as cross validation can be employed
- To search for the pruning branch, many technique has been proposed, such as the weakest-link cutting

# Advantages of the tree-structured approach

- Handles both categorical and ordered variables in a simple and natural way
- Automatic stepwise variable selection and complexity reduction
- It provides an estimate of conditional class probability
- It is invariant under all monotone transformations of individual ordered variables
- Robust to outliers and misclassified points in the training set
- Easy to interpret

# Issues

- Categorical covariate with $q$ classes produces $2^q - 1$ possible splits
  - Simplification is possible for binary classification with Gini index or cross entropy and regression with squared error loss
  - But it is unclear for multiclass classification (digit recognition example)
- Cost-sensitive (non-standard) classification with unequal misclassification costs
- Splits based on combined covariates
- Lack of smoothness (blockwise constant)
- High variance, constructed tree is very sensitive to sample

# Bagging and Random Forest

# Bagging

- Bootstrap aggregation, or bagging, is a general-purpose procedure for reducing the variance of a statistical learning method; we introduce it here because it is particularly useful and frequently used in the context of decision trees.
- Recall that given a set of n independent observations $Z_1, \ldots, Z_n$, each with variance $\sigma^2$, the variance of the mean $\overline{Z}$ of the observations is given by $\sigma^2/n$.
- In other words, averaging a set of observations reduces variance. Of course, this is not practical because we generally do not have access to multiple training sets.

# Bagging

1. Draw bootstrap samples $(x_1^{*b}, y_1^{*b}), \ldots, (x_n^{*b}, y_n^{*b})$; $b = 1, \ldots, B$
2. For each bootstrap sample, fit the tree model $\hat{f}^{*b}(x)$
3. The bagging estimate is

$$\hat{f}_{bag}(x) = \begin{cases} B^{-1} \sum_{b=1}^{B} \hat{f}^{*b}(x), & \text{for regression/classification;} \\ \text{mode}(\hat{f}^{*1}(x), \ldots, \hat{f}^{*B}(x)), & \text{for classification} \end{cases}$$

# Bagging classification trees

- The average of B predictions applied to regression trees
- For classification trees: for each test observation, we record the class predicted by each of the B trees, and take a majority vote: the overall prediction is the most commonly occurring class among the $B$ predictions.

# Out-of-Bag error estimation

- It turns out that there is a very straightforward way to estimate the test error of a bagged model.
- Recall that the key to bagging is that trees are repeatedly fit to bootstrapped subsets of the observations. One can show that on average, each bagged tree makes use of around two-thirds of the observations.
- The remaining one-third of the observations not used to fit a given bagged tree are referred to as the out-of-bag (OOB) observations.
- We can predict the response for the ith observation using each of the trees in which that observation was OOB. This will yield around B=3 predictions for the ith observation, which we average.
- This estimate is essentially the LOO cross-validation error for bagging, if $B$ is large.

# Why it works: some intuitions

- For regression, pretend we draw bootstrap sample from the true distribution rather than $\mathcal{T}$ (so independent),

$$\mathbb{E}(Y - \hat{f}(X))^2 \geq \mathbb{E}(Y - \hat{f}_{bag}(X))^2,$$

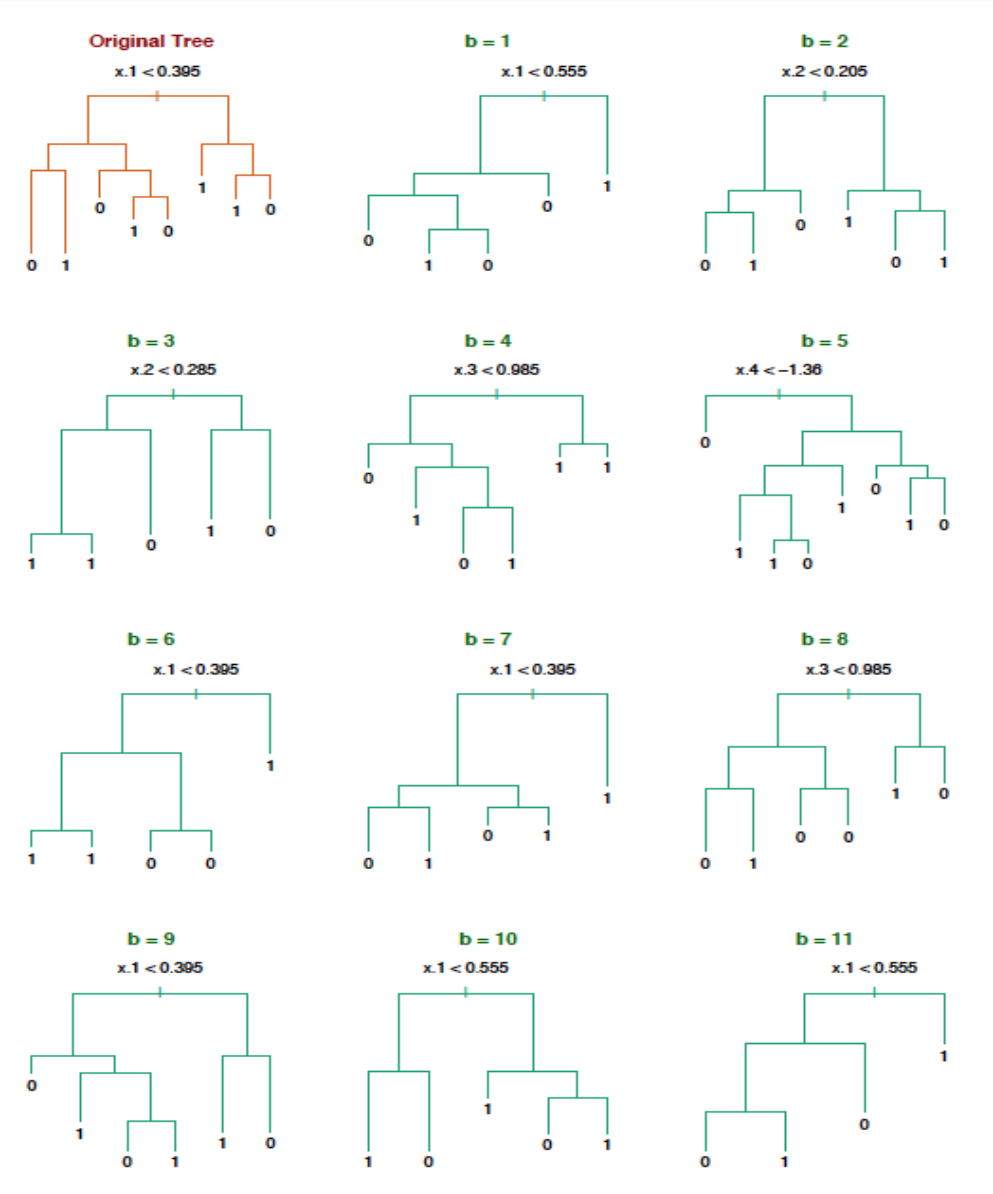
  as $\hat{f}_{bag}(x) = \mathbb{E}_{\mathcal{T}}(\hat{f}(x))$
- For classification, $\hat{f}_{bag}(x) \neq \mathbb{E}_{\mathcal{T}}(\hat{f}(x))$ and hence that the above inequality does not hold
  - "wisdom of crowds": the collective knowledge of a diverse and independent body of people exceeds the knowledge of any single individual
  - Two heads are better than one

# Random forest

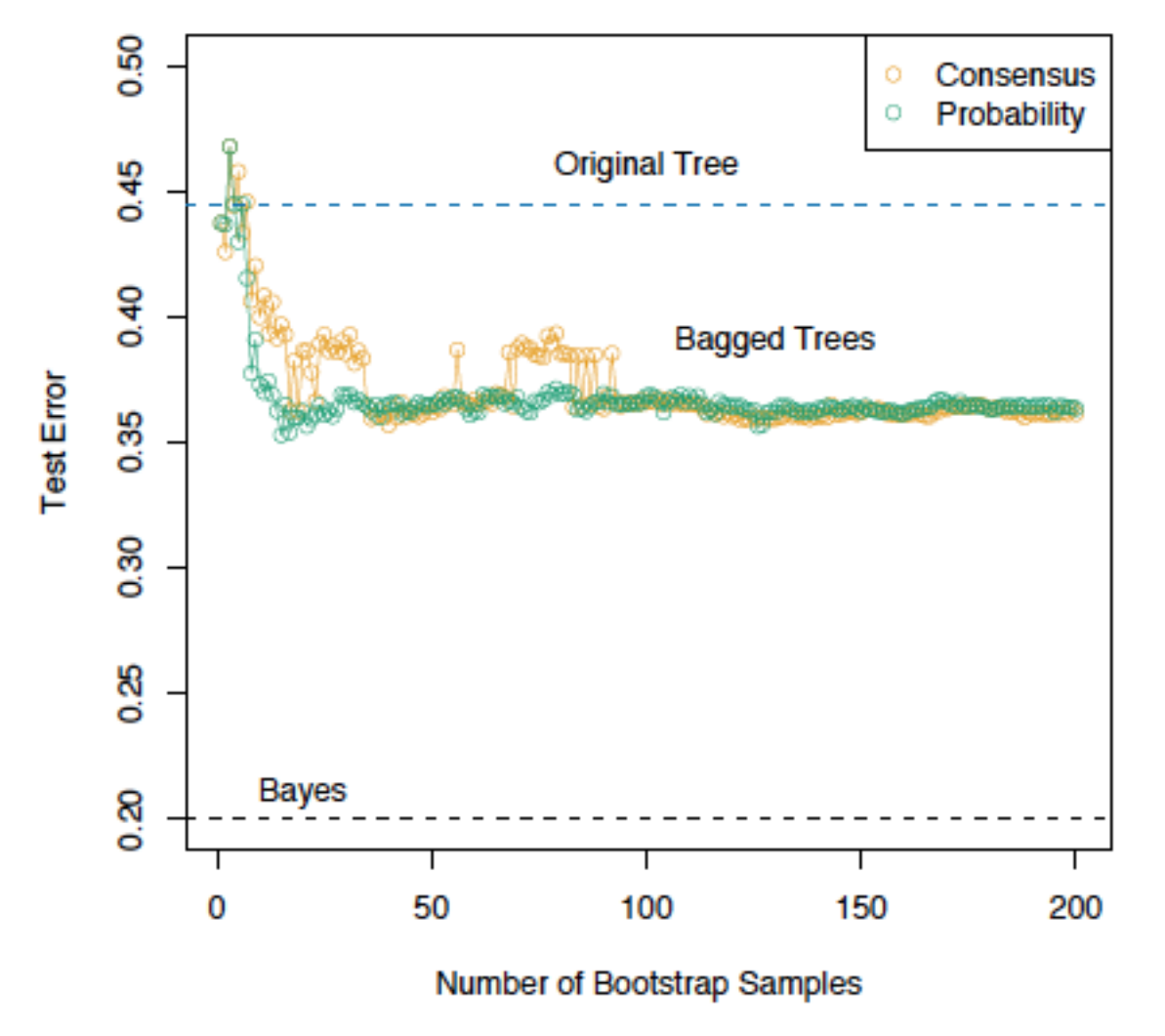
Random forest is similar to bagging, but different as it averages *de-correlated* trees

1. Draw bootstrap samples $(x_1^{*b}, y_1^{*b}), \ldots, (x_n^{*b}, y_n^{*b})$
2. For each bootstrap sample, grow a tree by repeating:
   a. randomly select a subset of the $p$ covariates
   b. pick the best split among the chosen subset
   c. split the node
3. The random forest estimate is constructed similarly as in bagging (average for regression, and majority vote for classification)

# A simulated examples



# A simulated examples



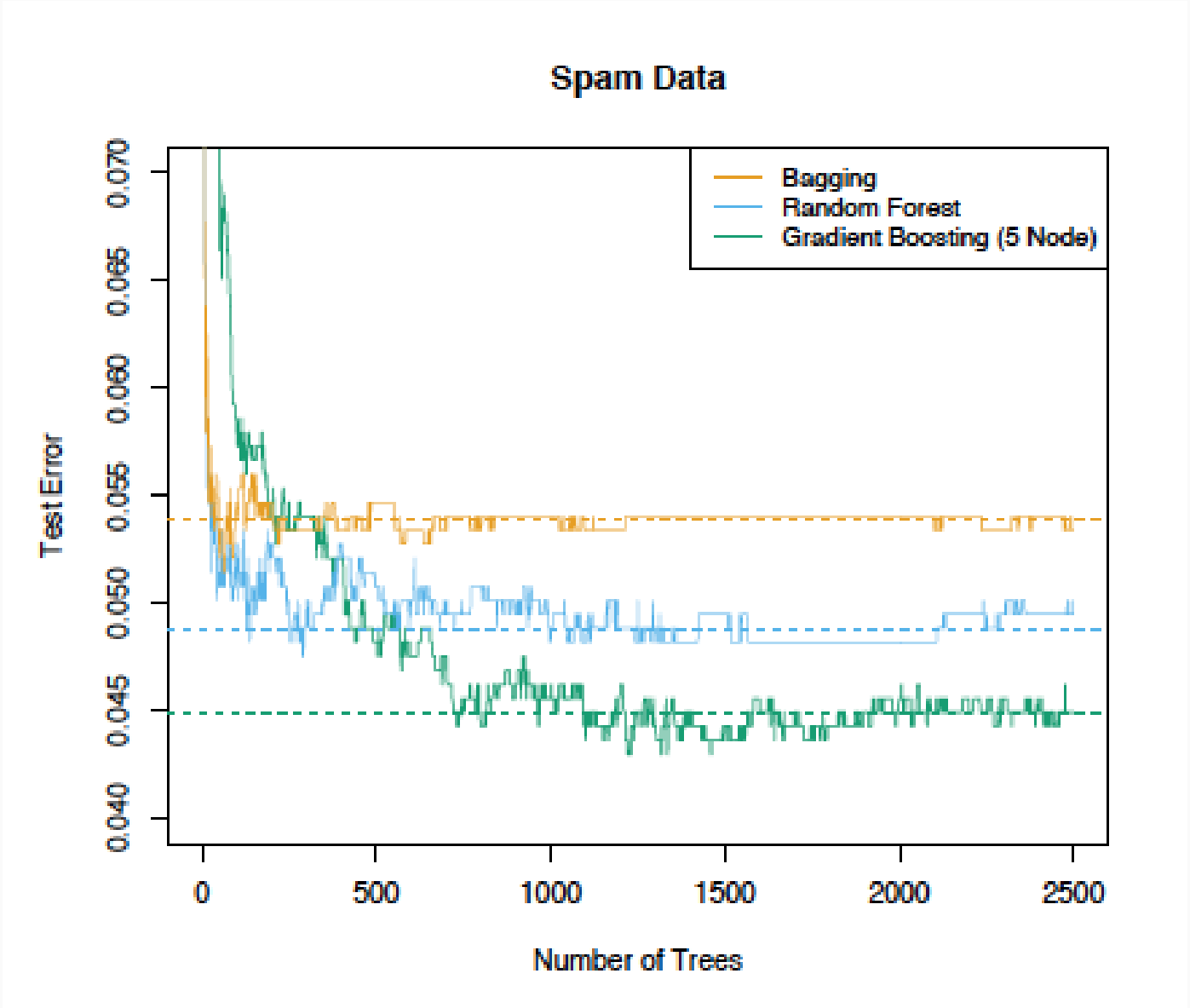
# Why it works: some intuitions

- In bagging, the bootstrap trees are correlated, and correlation limits the benefit of averaging
- Averaging $B$ variables with variance $\sigma^2$ and correlation $\rho$ yields variance

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

- In random forest, the correlation is reduced by selection of covariates in growing trees
- The price it has to pay is the increment of bias and variance, which, fortunately, is usually small

# Spam detection



# Additional readings

Breiman, L. (2001). Random forests. *Mach. Learn.*, **45**, 5-32.

Biau, G., Devroye, L. and Lugosi, G. (2008). Consistency of random forests and other averaging classifiers. *J. Mach. Learn. Res.*, **9**, 2039-2057.